

RISC-V VECTOR-ACCELERATED CANNY EDGE DETECTION PIPELINE

Technical Implementation Manual & Performance Analysis Documentation

Target Architecture:	RISC-V RV64GCV (Vector Extension Enabled)	Verification Framework:	GoogleTest (34 Functional Unit Tests)
Software Parity:	100% Bit-Exact SHA-256 Validated	Simulation Environment:	QEMU User-Mode Emulation

1. Executive Summary & System Overview

This document outlines the engineering architecture, structural implementation, and mathematical optimization methodologies developed for a high-performance **Canny Edge Detection Pipeline**. Tailored for embedded deployment within the **RISC-V** ecosystem, the pipeline incorporates both a standard scalar C++ code path (optimized for execution on `rv64gc` devices) and an accelerated vector parallelized path utilizing the native **RISC-V Vector Extensions (RVV)** targeting `rv64gcv` hardware builds.

A core project imperative was establishing an ultra-reliable, deterministic implementation capable of streaming standard matrix configurations. The complete execution pipeline runs seamlessly across five modular processing stages. Rigorous validation is continuously enforced via a suite of 34 unit tests implemented under the GoogleTest framework, while cross-compiled targets are comprehensively verified byte-for-byte using cryptographic SHA-256 hashing.

2. Pipeline Structural Architecture

The Canny Edge Detection framework processes a raw, single-channel 8-bit grayscale frame grid through five strict processing stages to accurately compute fine-grain edge boundaries while heavily suppressing environment ambient noise:

Stage 1: Gaussian Noise Filtering

To prevent high-frequency noise from corrupting edge estimation, a two-dimensional 5×5 Gaussian kernel convolves across the spatial grid. The implementation guarantees strict output boundary integrity using zero-padding boundary rules.

Stage 2: Spatial Intensity Gradient Generation (Sobel Operators)

Dual directional convolution operators are applied across the smoothed pixels. The horizontal derivative (G_x) and vertical derivative (G_y) are extracted concurrently via parallel row-column kernel passes:

G_x = Matrix Convolved with $[[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]$

G_y = Matrix Convolved with $[[-1, -2, -1], [0, 0, 0], [1, 2, 1]]$

Stage 3: Gradient Magnitude Compilation & Quantization

To guarantee embedded speedups, the vector magnitude is evaluated using the L1-Norm approximation to bypass hardware-intensive square root calculations. The directional orientations are simultaneously mapped to localized quadrants representing standard angles: 0° , 45° , 90° , and 135° .

Stage 4: Non-Maximum Suppression (NMS)

The magnitude maps are thinned into ultra-sharp 1-pixel wide local maximum lines. For every target coordinate, the central gradient magnitude is contrasted against its two immediate neighbors aligned precisely along the local quantized direction vector. If the central magnitude value is not strictly greater than both neighbors, it is suppressed to zero.

Stage 5: Double Thresholding & Hysteresis Continuity Tracing

Pixels are classified via a dual-threshold system. Pixels exceeding the *High Threshold* are marked as permanent strong edges. Pixels dropping below the *Low Threshold* are discarded. Intermediate pixels are classified as weak edges, and are dynamically promoted only if they share direct 8-way spatial connectivity with an adjacent strong pixel.

3. Algorithmic and Mathematical Optimizations

Standard Canny implementations suffer severe compute bottlenecks on low-power embedded processors due to heavy reliance on floating-point functions such as `atan2()`, `sqrt()`, and floating-point divisions. This architecture introduces specific integer arithmetic optimizations:

Branchless Integer Cross-Multiplication for Angle Mapping

Directional quantization traditionally evaluates $\theta = \text{atan2}(G_y, G_x)$. This implementation completely eliminates transcendental trigonometric calculations by substituting them with absolute value evaluations and integer cross-multiplication scaling factors. The geometric boundaries separating the quadrants are represented as rational fixed-point fractions:

$$0^\circ \text{ Sector: } |G_y| \times 1000 \leq |G_x| \times 414$$

$$90^\circ \text{ Sector: } |G_y| \times 1000 > |G_x| \times 2414$$

$$45^\circ / 135^\circ \text{ Sectors: Allocated depending on the matching sign parity: } \text{sign}(G_x) == \text{sign}(G_y)$$

Optimization Impact:

By avoiding floating-point trigonometry, the scalar direction mapping module drops raw execution cycle demands significantly, making it perfectly fitted for fixed-point integer pipelines.

4. RISC-V Vectorization (RVV) Strategy

The parallel vectorized pipeline path leverages the **RISC-V Vector Extension v1.0** assembly intrinsics. The codebase is engineered to be entirely vector-length agnostic (VLEN-agnostic), ensuring seamless scalability whether deployed on 128-bit, 256-bit, or 512-bit wide hardware vector units.

Core Vector Design Patterns Applied:

- **Strip-Mining Loops:** Vector loop iterations utilize the `vsetvl` instruction family to dynamically query the optimal vector register capacity (`vl`) per iteration pass, gracefully processing arbitrary image column lengths.
- **Narrowing Chains & Data Packings:** Memory bandwidth is minimized by maintaining a strict, tightly bound data lifecycle. Inputs are stored as standard 8-bit unsigned matrices (`uint8_t`). Intricacies within the high-magnitude Sobel intermediate additions scale upward into 16-bit signed registers (`int16_t`) to prevent overflow conditions, before executing a narrowing down-sample chain back into packed 8-bit spaces.
- **Vector Integer Normalization:** Fixed-point scale normalization for the 5×5 Gaussian filter (summing to 273) is performed using exact vector integer divisions, ensuring completely identical bit-level parity with the scalar pipeline.

```
// Architectural Guarding Pattern for Safe Multi-Platform Compositions
#ifdef __riscv_vector
    // Highly-optimized RISC-V Vector Intrinsic Core Path Execution Loop
    size_t vl = __riscv_vsetvl_e16m2(width, fflags);
    vint16m2_t vx_grad = __riscv_vle16_v_i16m2(ptr_x, vl);
    vint16m2_t vy_grad = __riscv_vle16_v_i16m2(ptr_y, vl);
#else
    // Fallback Robust Scalar Core Execution Path
#endif
```

5. Verification Methodology & Performance Analysis

Software validation and architectural performance profiling were carried out across multiple target platforms under QEMU user-mode emulators.

Functional Validation & Parity Report

The compilation pipeline mandates a rigorous regression testing environment. 34 specialized GoogleTest unit tests evaluate functional components independently under challenging input stresses. Cross-compilation validation requires absolute binary output mirroring:

Validation Stage Target	Scalar Code Path Output	RISC-V Vector Path Output	Verification Status
Gaussian Noise Filter Output	SHA-256 Verification Pass	SHA-256 Verification Pass	Byte-Identical (Match)
Sobel Spatial Gradients	SHA-256 Verification Pass	SHA-256 Verification Pass	Byte-Identical (Match)
Quantized Direction Matrices	SHA-256 Verification Pass	SHA-256 Verification Pass	Byte-Identical (Match)
Final Suppressed Edge Frame	SHA-256 Verification Pass	SHA-256 Verification Pass	Byte-Identical (Match)

Profiling Breakdown (Scalar Execution Path)

Execution diagnostics derived across 100 iterations on a standard 256×256 image frame provide a distinct perspective regarding computational workload distribution:

- **Gaussian Convolution Filter:** 38.70% total runtime allocation (1976.56 ms)
- **Sobel Directional Gradients:** 57.56% total runtime allocation (2939.38 ms)
- **Gradient Magnitude Evaluation:** 2.39% total runtime allocation (122.29 ms)
- **Direction Quantization:** 1.34% total runtime allocation (68.65 ms)

Critical Note regarding QEMU Emulation Behavior:

When executing binaries inside the QEMU emulator layer, the software-emulated RVV vector paths reflect an artificial performance penalty compared to the native scalar compilation. This is a known architectural artifact: QEMU translates RISC-V Vector instructions via software loops rather than utilizing underlying host execution hardware. Thus, emulated profiling is utilized exclusively to assert **functional correctness** and VLEN agnostic safety, rather than evaluating raw native physical hardware speedups.

6. Environment Setup & Execution Guidelines

The repository includes a comprehensive `Makefile` alongside utility scripts to fully automate cross-compilation, asset pre-processing, and verification checks.

Primary Build Commands

```
# Compile the entire project suite for native host development and testing
make

# Execute the GoogleTest functional validation suite (34 test cases)
make test

# Execute cross-compiled scalar vs vector parity verification
make verify

# Run benchmarking runs for scalar and vector pipelines under QEMU
make benchmark
```

Image Preprocessing Layer (fix_image.py)

To accommodate strict embedded memory configurations, a Python preprocessing script automated via the `Pillow` library converts incoming arbitrary image assets into flattened 8-bit raw grayscale byte arrays reshaped precisely to a standardized **256×256** resolution grid layout. This protects execution boundary pointers from undefined segment out-of-bound errors during vector register processing.